

2.5 grep Family: Regular expressions, grep, egrep, fgrep, tr.

Various File Processing Commands:

The head command

You can use the head command to look at the beginning of a file.

Syntax:

head <option><filename>

e.g.

\$ head first.text

By default, you can only read the first ten lines of a file. You can change the number of lines to be displayed by specifying a number option, as shown in the following command:

Option:

Option	Description
n	Display specified number of lines

e.g.

\$ head -20 temp

In this command, you can view first 20 lines of a temp file

The tail command

The reverse of head is tail command, Using tail, you can view the last ten lines of a file.

Syntax

Tail option <filename>

e.g.\$ tail first.text

By default, you can read the only last ten lines of a file. You can change the number of lines to be displayed by specifying a number of option.

Options	Description
-n	Display specified number of lines
+n	Display all lines beginning from nth line

e.g.

\$: tail-20temp

In this command , you can view last 20 lines of temp file.

The tr command

The `tr` command is used to translate one set of character to another. This command can also

used to squeeze repeated character into one. Several commands have multi-column output and gap between columns is more than one space. In such cases, if user wants particular

column, `cut` cannot be used, since the column separator has to be used single character. Therefore,

the solution is to squeeze the several spaces between columns into a single space, and then use

`cut` command to extract the desired columns.

The `-s` option is used to specify squeezing of several occurrences of a character into One character

E.g. `$ who <enter>`

```
Param      tty2  june  1  9:06
```

```
Prashant   tty3  june  1 11:20
```

```
$
```

To make a column separator a single space, `tr -s` has to be used as shown

```
$ who > temporary <enter>
```

```
$ tr -s " " temporary <enter>
```

```
Param tty2 june 1 9:06
```

```
Prashant tty3 june1 11:20
```

Here the `-s` option of `tr` works on every record, squeezing , occurrences of string given In quotes(single space, in this case) to one space.

e.g . `$ echo abcdefaabbcc ! tr 'a-c' '1-3' <enter>`

```
123def112233
```

In this example, `echo` sends the letter `abcdefaabbcc` to `tr` command `tr` looks up each letter in its

first list if found, it translates it to the corresponding character in its second list. If the character is not found in first list, it is passed "as is" to the output

Another common usage of `tr` is case conversion.

```
$ tr "[a-z]" "[A-Z]" <enter>
```

```
CUSROW WADIA INSTITUTE OF TECHNOLOGY,PUNE
```

```
$
```

The above command converts all lowercase to uppercase.

The cut command

The cut command of unix/ linux is useful when specific columns from the specific file need to be extracted

Table for option of cut

Options	Function
-f	Display the fields specified
-c	Cutting of columns i.e. character b character
-d	Specifies the column delimiter

The -f options can be used to specify a field number if a tab character field separator.

e.g

```
$ cat data 1
```

```
Vishal 1979 2.10
```

```
Ladkat 1980 15.0
```

```
Param 1978 17.4
```

```
$ cut -f1, 3 data 1
```

```
Vishal 2.10
```

```
Param 17.4
```

```
$
```

In this example,-f1,3 tells cut out and keep the fields 1 and 3

e.g. \$ cat data2

```
param suresh jaldawar
```

```
$ cut -d `` -f1,3data2
```

```
Param jaldawar
```

```
$
```

The paste command

Paste command is used to paste several files together

Syntax

Paste [-s] [-d char]file1 file2....

\$cat f1 (file1 consider as f1)

a

b

c

d

```
$ cat f2 (file 2 consider as f2)
```

```
1  
2  
3  
4  
5  
6
```

```
$ paste f1 f2
```

```
a 1  
b 2  
c 3  
d 4  
 5  
 6
```

```
$
```

The join command

The join command enables you to merge data from a line in one file with data from corresponding

Line in second file on the basis of a key field that contains entries common to both of them.

The join command requires that both input files be stored according to the common field on which

They are joined.

Consider example

Here is file for day1 contain following data

```
$cat day1
```

```
ABC 2  
BCD 24  
DEF 6
```

```
$cat day2
```

```
ABC 2  
BCD 12  
DEF 22
```

```
$ join day1 day2
ABC  2  2
BCD 24 12
DEF  6 22
$
```

User can use other fields as the common field with the `-j` option.

e.g.

```
$join -j12 -j24 file1 file2
```

The split command

Split command is used to split file into smaller pieces. If file become too large, split lets user to

Handle the file in more manageable pieces.

Syntax: `split -number file tagname`

`-number` = specifies the number of lines to include in each file.

File = The file to split into smaller pieces

Tagname=by default, split builds the output creating the files named: xaa, xab, xac
And so on.

e.g.

```
$ split -500 studentlst newlist
```

The sort command

Many times you want to arrange contents of a file in specific order. Whether it is comparing files in alphabetic order or setting up database in numeric order Linux features a particularly useful tool for sorting files, named sort

Syntax

```
Sort <options> file1 file2 file3
```

Table for option of sort command

Command	Function
-tchr	Uses delimiter chart to identify fields
-n	Sorts numerically
-r	Reverses sort order
-c	Checks if file is sorted
-o fname	Places o/p in file fname

-u	Remove duplicates records
-m.n	Stops sort on nth column of (m+1)th field
+m.n	Starts sort after nth column of (n+1)th field
-k	Stop sort after kth field
+k	Start sort after skipping kth field
-m list	Merges sorted files in list

```
e.g.  $ cat tstfile
      Zebra
      Monkey
      Apple
      $
```

To ignore leading spaces in line when sorting use -b option

```
$ sort -b tstfile
Apple
Monkey
Zebra
$
```

```
e.g  $ sort numfile
12      Param
2       Vishal
5       Jaldwar
66      Suresh
$
$ sort -n numfile
2       vishal
5       jaldwar
12      param
66      suresh
$
```

In this example file is sorted on basis on of magnitude of number at start of line

The comm command

Comm command compares two sorted files. It compares each line of the first file with it's corresponding line in the second file. (Uniqueness & commonness)

It displays output in three columns,

lines unique to the first file,
lines unique to the second file and
lines common to both files, respectively.

It can be used to show any of the columns (Uniqueness & commonness) in the output. To drop a particular column, simply use its column number as an option prefix along with `-'.

Syntax: comm file1 file2

```
$cat file1
```

```
Diya
```

```
Prajakta
```

```
Adhirath
```

```
$cat file2
```

```
Vinayak
```

```
Prajakta
```

```
Abhishek
```

```
Akanksha
```

```
$comm file1 file2
```

```
Diya
```

```
        Vinayak
```

```
                Prajakta
```

```
        Abhishek
```

```
Adhirath
```

```
        Akanksha
```

```
$
```

The cmp command

The cmp command is used to compare files byte by byte when used without options , It uses two file names as arguments. It displays the first difference on the terminal. If two files are identical, cmp displays no message, but simply returns the prompt. (Difference)

Syntax: cmp file11 file22

The option -l(list) gives detailed list of

the byte no. and

the differing byte in overall for each character that differ in both the files.

```
$cat file11
```

This is Linux Operating System

```
$cat file22
```

This is Windows Operating System

```
$ cmp file11 file22
```

file11 file22 differ: byte 9, line 1

```
$
```

The diff command

To show differences between two text files also it tells you which lines in one file have to be changed to make the two files identical. Diff used certain special symbols and instructions to indicate the changes that are required to make two files identical.

a: add

c: change

d: delete

Syntax: diff file1 file2

```
$cat file1
```

Diya

Prajakta

Adhirath

```
$cat file2
```

Vinayak

Prajakta

Abhishek

Akanksha

```
$diff file1 file2
```

1c1

< Diya

```
---
>Vinayak
3c3,4
< Adhirath
---
Abhishek
Akanksha
```

The wc command

Linux features a universal word count program. It counts **no of lines, words and characters**, depending on the options used. It takes one or more files as it's arguments and displays a four column o/p.

e.g `$ wc infile`
`3 20 103 infile`
Wc counts 3 lines, 20 words and 103 characters.

It offers 3 option to make a specific count.

Options	Function
-l	Counts only the number of lines
-w	Counts only the number of words
-c	Counts only the number of characters

The more command

If file is too large for it's contents to fit in one screen, it will scroll off your screen when you view it with cat. More allows the user to view a file, one screen at a time.

e.g. `more chap01`

You will see the contents of chap 1 on the screen , one page at a time.

Option	Function
Spacebar	Scrolls forward one screen
F	Scrolls forward one screen
B	Scrolls backward one screen
J	Scrolls forward one line
K	Scrolls backward one line
200G	Goes to line no. 200
G	Goes to last line of file

The pg command

This command is used to display large file, a screen at a time

Syntax

Pg filename <enter>

\$ pg data1

The above command will display the contents of data1, a screen at a time

The grep command

grep is the most powerful filters, it scans the file for the occurrence of a pattern, and can display the selected pattern.

Syntax: grep options pattern filename(s)

Grep requires expression to represent the pattern to be searched for, followed by one or more filenames.

Options	Function
-c	Displays count of number of occurrence
-l	Displays list of filenames
-n	Displays line numbers along with lines
-v	Displays all but lines matching expression
-i	Ignores case for matching
-h	Omits filenames when handling multiple files

The difference between grep, egrep and fgrep

One of the renowned search tool on Unix-like systems which can be used to search for anything whether it be a file, or a line or multiple lines in file is grep utility. It is very vast in functionality which can be attributed to the large number of options it supports like: searching using string pattern, or reg-ex pattern or perl based reg-ex etc.

Due its varying functionalities, it has many variants including **grep**, **egrep** (**Extended GREP**), **fgrep** (**Fixed GREP**), **pgrep** (**Process GREP**), **rgrep** (**Recursive GREP**) etc. But these variants have minor differences to original **grep** which has made them popular and to be used by various Linux programmers for specific tasks.

Main thing that remains to be investigated is what are the differences between the three main variants i.e. **'grep'**, **'egrep'** and **'fgrep'** of grep that makes Linux users choose one or the other version as per requirement.

Some Special Meta-Characters of grep

1. **+** – Equivalent to one or more occurrences of previous character.
2. **?** – This denotes almost 1 repetition of previous character. Like: **a?** Would match **'a'** or **'aa'**.
3. **(** – Start of alternation expression.
4. **)** – End of alternation expression.
5. **|** – Matching either of the expression separated by '|'. Like: **"(a|b)cde"** would match either **'abcde'** or **'bbcde'**.
6. **{** – This meta-character indicates start of range specifier.
Like: **"a{2}"** matches **"aa"** in file i.e. a 2 times.
7. **}** – This meta-character indicates end of range specifier.

The e in egrep stands for extended and the f in fgrep stands for fixed.

- egrep and fgrep are deprecated commands and should be avoided.
- egrep is equivalent to grep -E command and it allows the use of extended regex.
- fgrep is equivalent to grep -F command and it uses a fixed string for search and hence it performs a faster search.

Standard grep command

This is the standard version of grep designed for searching patterns using basic regular expressions. Using grep requires escaping meta-characters to generate the expected behavior.

Suppose I want to search for either tast or test. This is what I can use:

```
christopher@linux-handbook:~$ grep 't\(a\|e\)st' test.txt
```

Extended Grep: Egrep or grep -E

Egrep was created to provide **extended** support for meta-characters. This means that it escapes these special characters by default. This saves time and generally makes the expressions easier to read.

The command I used above can be re-written as the much more legible:

```
christopher@linux-handbook:~$ egrep 't(a|e)st' test.txt
```

The `egrep` binary does not consistently support the `{` character. This can result in unexpected behaviors.

Some versions of `egrep` allow them to be interpreted properly as literal characters if escaped (`\{`). However, the best practice for matching a literal `{` is to enclose in square brackets `[{]`.

fgrep Command

fgrep or the **Fixed grep** or **grep -F** is yet another version of `grep` which is fast in searching when it comes to search for the entire string instead of regular expression as it doesn't recognize the regular expressions, neither any meta-characters. For searching any direct string, this is the version of `grep` which should be selected. `fgrep` searches for complete string and doesn't even recognize special characters as part of regular expression even if escaped or not escaped.

```
$ fgrep -C 0 '(f|g)ile' check_file
```

```
$ fgrep -C 0 '\(f|g\)ile' check_file
```

Like, when meta-characters were not escaped, `fgrep` searched for the complete string `"(f|g)ile"` in the file, and when the meta-characters were escaped, then the `fgrep` command searched for `"\ (f|g\) ile"` all characters as is in the file.